

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



BÁO CÁO TUẦN 11

“ Xây dựng Website đặt sân thể thao tích hợp AI chatbot ”

Giảng viên hướng dẫn: Kim Ngọc Bách

Họ và tên sinh viên : Bùi Trung Đức

Mã sinh viên : B23DCCN166

Lớp : D23CQCN12-B

Nhóm học phần : 11

Hà Nội, 5/2026

I. Công việc đã thực hiện trong tuần

1. Sửa Backend quản lí đặt sân bên admin

a. Phân truy vấn các booking.

File: *BookingDAO.java*

```
public interface BookingDAO extends JpaRepository<Bookings, Integer> {

    @Query(value = "SELECT \r\n" + "      b.bookingid,\r\n" + "      b.bookingdate,\r\n" + "      b.bookingprice,\r\n" + "      b.note,\r\n" + "      b.bookingstatus,\r\n" + "      f.namefield,\r\n" + "      f.image\r\n" + "    bookings AS b\r\n" + "  JOIN \r\n" + "      bookingdetails AS bd ON b.bookingid = bd.bookingid\r\n" + "  JOIN \r\n" + "      field AS f ON bd.fieldid = f.fieldid\r\n" + " WHERE \r\n" + "      b.username = :username ORDER BY bookingdate DESC LIMIT 20", nativeQuery = true)
    List<Object[]> getBookingInfoByUsername(String username);

    @Query(value = "SELECT \r\n" + "      b.bookingid,\r\n" + "      b.bookingdate,\r\n" + "      b.bookingstatus,\r\n" + "      bd.shiftid,\r\n" + "      bd.playdate,\r\n" + "      bd.price,\r\n" + "      f.namefield,\r\n" + "      f.image,\r\n" + "      s.nameshift\r\n" + " FROM \r\n" + "      bookings AS b\r\n" + "  JOIN \r\n" + "      bookingdetails AS bd ON b.bookingid = bd.bookingid\r\n" + "  JOIN \r\n" + "      field AS f ON bd.fieldid = f.fieldid\r\n" + "  JOIN \r\n" + "      shifts AS s ON bd.shiftid = s.shiftid\r\n" + " WHERE \r\n" + "      b.bookingid = :bookingid", nativeQuery = true)
    Object[] getBookingInfoByBookingDetail(String bookingid);

    @Query(value = "select count(*) from bookings", nativeQuery = true)
    int countBooking();
    // admin
    @Query(value = "SELECT b.* FROM bookings b \r\n" + "      JOIN users u ON b.username = u.username where b.bookingstatus like '%Đã Cọc%' and date(b.bookingdate) <= :date", nativeQuery = true)
    List<Bookings> findAllBookingAndUser();
    // search admin
    @Query(value = "SELECT b.* FROM bookings b " + "      JOIN users u ON b.username = u.username " + " WHERE (CONCAT(u.firstname, ' ', u.lastname) LIKE %:keyword%) " + " AND b.bookingdate LIKE %:datebook% " + " AND b.bookingstatus LIKE %:status%", nativeQuery = true)
    List<Bookings> findByConditions(@Param("keyword") String keyword,
                                   @Param("datebook") Date datebook,
                                   @Param("status") String status);
```

findAllBookingAndUser() — lấy danh sách booking hiện tại kèm thông tin user

findByConditions() — tìm kiếm booking theo từ khóa, ngày, trạng thái

```

+ "WHERE bookingstatus = 'Hủy Đặt' AND date(bookingdate) = curdate(), , nativeQuery = true)
List<Object[]> tongKeBookingTrongNgay();

// tổng số phiếu đặt sản phẩm tháng này và tháng trước
@Query(value = "SELECT 'This Month' AS period, COUNT(*) AS total bookings\r\n" + "FROM bookings\r\n"
+ "WHERE YEAR(bookingdate) = YEAR(CURDATE()) AND MONTH(bookingdate) = MONTH(CURDATE())\r\n"
+ "UNION ALL\r\n" + "SELECT 'Last Month', COUNT(*)\r\n" + "FROM bookings\r\n"
+ "WHERE YEAR(bookingdate) = YEAR(CURDATE() - INTERVAL 1 MONTH) AND MONTH(bookingdate) = MONTH(CURDATE() - INTERVAL 1 MONTH)"
List<Object[]> tongSoPhieuDatSan2Thang();
// tổng doanh thu booking tháng này và tháng trước
@Query(value = "SELECT\r\n"
+ "    COALESCE(SUM(CASE \r\n"
+ "        WHEN bookingstatus = 'Hoàn Thành' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE) THEN bookingprice \r\n"
+ "        WHEN bookingstatus = 'Đã Cọc' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE) THEN (bookingprice * 0.3 )\r\n"
+ "        WHEN bookingstatus = 'Hủy Đặt' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE) THEN (- (bookingprice * 0.3 * 2) )\r\n"
+ "    END), 0) AS doanh_thu_thang_hien_tai,\r\n"
+ "    COALESCE(SUM(CASE \r\n"
+ "        WHEN bookingstatus = 'Hoàn Thành' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE - INTERVAL 1 MONTH) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE - INTERVAL 1 MONTH) THEN bookingprice \r\n"
+ "        WHEN bookingstatus = 'Đã Cọc' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE - INTERVAL 1 MONTH) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE - INTERVAL 1 MONTH) THEN (bookingprice * 0.3 )\r\n"
+ "        WHEN bookingstatus = 'Hủy Đặt' AND \r\n"
+ "            YEAR(bookingdate) = YEAR(CURRENT_DATE - INTERVAL 1 MONTH) AND \r\n"
+ "            MONTH(bookingdate) = MONTH(CURRENT_DATE - INTERVAL 1 MONTH) THEN (- (bookingprice * 0.3 * 2) )\r\n"
+ "    END), 0) AS doanh_thu_thang_truoc\r\n"
+ "FROM bookings;", nativeQuery = true)
List<Object[]> tongDoanhThuBooking2Month();

```

- Bổ sung thêm Query thống kê doanh thu theo tháng này và tháng trước
- b. Endpoint REST dùng để admin thao tác booking:

File: *BookingRestController.java*

```

MessageSource messagesource;
@Autowired
BookingDAO bookingDAO;

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationException(MethodArgumentNotValidException ex) {
    return GlobalExceptionHandler.handleValidationException(ex);
}

@GetMapping("getAll")
public ResponseEntity<List<Bookings>> getAll(Model model){
    return ResponseEntity.ok(bookingDAO.findAllBookingAndUser());
}

@GetMapping("get/{id}")
public ResponseEntity<Bookings> getOne(@PathVariable("id") Integer id) {
    if(!bookingDAO.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(bookingDAO.findById(id).get());
}

@PutMapping("update/{id}")
public ResponseEntity<Bookings> update(@PathVariable("id") Integer id, @Valid @RequestBody Bookings booking) {
    if(!bookingDAO.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    bookingDAO.save(booking);
    return ResponseEntity.ok(booking);
}

// search
@GetMapping("search")
public ResponseEntity<List<Bookings>> search(@RequestParam("keyword") String keyword, @RequestParam("datebook")
    return ResponseEntity.ok(bookingDAO.findByConditions(keyword, datebook, status));
}

```

- *GET /rest/bookings/getAll*: Dùng để lấy danh sách tất cả booking mà admin cần quản lý:
 - + Thực chất backend gọi *bookingDAO.findAllBookingAndUser()*.
 - + Query trong BookingDAO trả về các Bookings:join với bảng users
 - + Chỉ lấy booking có bookingstatus chứa Đã Cọc
 - + Chỉ lấy booking của ngày hiện tại (*date(b.bookingdate) = curdate()*)
 - + Kết quả trả về là *List<Bookings>* theo cấu trúc entity Bookings.
 - *GET /rest/bookings/get/{id}*:
 - + Dùng để lấy thông tin chi tiết một phiếu đặt sân theo bookingid.
 - + Flow:kiểm tra *bookingDAO.existsById(id)*, nếu không tồn tại thì trả 404 Not Found; nếu tồn tại thì trả *bookingDAO.findById(id).get()*
 - + Trả về object Bookings đầy đủ dữ liệu trong bảng bookings.
 - *PUT /rest/bookings/update/{id}*: Dùng để cập nhật thông tin booking (do admin sửa trạng thái, ghi chú, v.v.).
 - + Flow:kiểm tra *bookingDAO.existsById(id)*
 - + Nếu không tồn tại thì trả 404 Not Found; nếu tồn tại thì *bookingDAO.save(booking)*
 - + Endpoint này không lấy dữ liệu chi tiết từ path và dto riêng; nó save trực tiếp object Bookings đã gửi.
- Do đó client cần gửi đủ thông tin entity hoặc ít nhất phần cần cập nhật nếu entity mapping cho phép.

- *GET /rest/bookings/search*: Dùng để tìm kiếm booking theo điều kiện admin muốn.
- + keyword — tìm theo tên người dùng (CONCAT(u.firstname, ' ', u.lastname))
- + datebook — tìm theo ngày booking
- + status — tìm theo trạng thái booking
- + Backend gọi *bookingDAO.findByConditions(keyword, datebook, status)*.
- + điều kiện: họ tên user chứa keyword; bookingdate chứa datebook; bookingstatus chứa status

2. Sửa Backend lịch sử đặt sân người dùng

```
// Lịch sử đặt sân
@GetMapping("/field/profile/historybooking")
public String viewHistoryField(Model model) {
    List<Object[]> listbooking = bookingservice.getBookingInfoByUsername(userlogin); // Danh sách booking
    model.addAttribute("listbooking", listbooking);
    return "user/lichsudatsan";
}

// Chi tiết lịch sử đặt sân
@GetMapping("/field/profile/historybooking/detail")
public String viewDetail(Model model, @RequestParam("bookingId") String bookingId,
    @RequestParam("bookingPrice") double bookingPrice) {
    double giamgia = 0.0; // giảm giá
    double phuthu = 0.0; // Phụ thu
    double tiencoc = 0.0; // tiền cọc
    double tamtinh = 0.0; // tạm tính
    double conlai = 0.0; // tiền còn lại
    Object[] listbookingdetail = bookingservice.getBookingInfoByBookingDetail(bookingId); // Chi tiết ph
    for (Object object : listbookingdetail) {
        if (object instanceof Object[]) { // kiểm tra phần tử hiện tại có phải đối tượng object hay không
            Object[] arrayObject = (Object[]) object; // ép kiểu phần tử hiện tại thành mảng đối tượng
            if (arrayObject.length >= 9) { // kiểm tra xem mảng đối tượng có ít nhất 9 đối tượng
                int shiftid = (int) arrayObject[3]; // trích giá trị thứ 3
                double price = (double) arrayObject[5]; // trích giá trị thứ 5
                if (shiftid > 10 || shiftid < 3) { // điều kiện có phụ thu
                    phuthu = price * 30 / 100;
                    tamtinh = phuthu + price;
                    giamgia = bookingPrice - tamtinh;
                    tiencoc = bookingPrice * 30 / 100;
                    conlai = bookingPrice - tiencoc;
                } else {
                    tamtinh = price;
                    giamgia = bookingPrice - tamtinh;
                    tiencoc = bookingPrice * 30 / 100;
                    conlai = bookingPrice - tiencoc;
                }
            }
        }
    }
}
```

- *bookingservice.getBookingInfoByUsername(userlogin)* lấy dữ liệu lịch sử booking của user đang đăng nhập.
- Kết quả được lưu vào listbooking và đưa vào model với tên *listbooking*.
- Cuối cùng trả về view *user/lichsudatsan* để hiển thị danh sách lịch sử đặt sân
- *bookingservice.getBookingInfoByBookingDetail(bookingId)* lấy chi tiết booking theo id.
- Kết quả được gán vào *Object[] listbookingdetail*.
- Dòng for (*Object object : listbookingdetail*) duyệt từng phần tử trong mảng trả về.

- Nếu phần tử đó là Object[], code tiếp tục xử lý: *arrayObject[3]* lấy shiftid (mã ca)
- *arrayObject[5]* lấy price (giá ca)

II. Kế hoạch tuần tiếp theo

- Nghiên cứu các kiến thức về API để tích hợp AI vào Website
- Triển khai tích hợp chatbox AI vào web, thiết kế UI/UX